

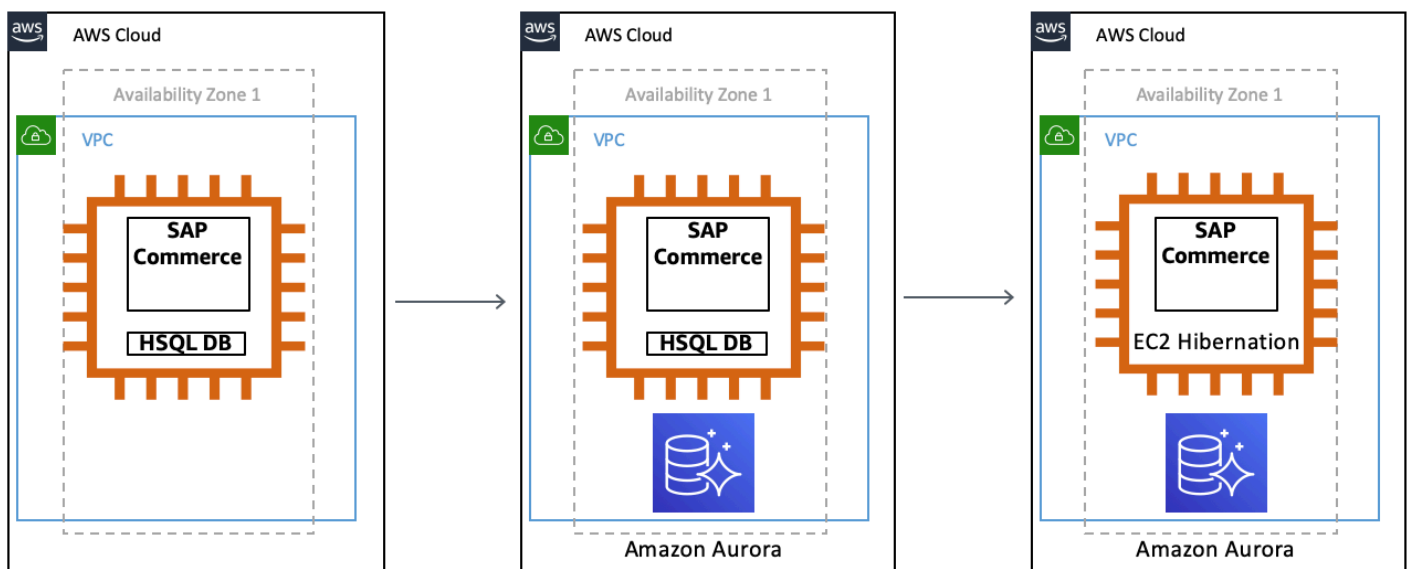
Driving new levels of agility for your SAP workloads: an example with SAP Commerce

by Patrick Leung | on 18 JUN 2020 | in [Amazon Aurora](#), [Intermediate \(200\)](#), [SAP on AWS](#), [Technical How-to](#) | [Permalink](#) | [Share](#)

For a long time, on-premises SAP projects involved lots of upfront requirement analyses, evaluation, and sizing processes to minimize risk. In practice, we've encountered many situations where, after weeks of planning, those initial decisions will have to change. Your SAP architecture must be flexible enough to respond quickly to business demand, implement solutions quickly, refine based on feedback, and repeat. In this blog post, we use SAP Commerce as an example to demonstrate how you can have an agile SAP architecture on AWS. We start with a minimum lovable product (MLP), then walk you through how you can pivot your architecture quickly.

SAP Commerce Overview

Let's start with a quick introduction to SAP Commerce. Most readers probably recognize the product's predecessor, SAP Hybris, which is a popular omnichannel e-commerce platform. In 2017, we announced the [certification of Amazon Aurora for SAP Hybris](#). In 2018, SAP Hybris was rebranded to SAP Commerce and is now part of the SAP customer experience (CX) suite.



We start with an architecture that uses HSQLDB and the gradle scripts that come pre-bundled with SAP Commerce. Then, we show you how easy it is to iterate on this architecture. This is a great way to experiment with SAP Commerce on AWS, and have a running system on AWS in about 60 minutes.

The first iteration is changing the database. HSQLDB is suitable for a demo environment, but customers typically move to a commercial-grade database once they move out of the demo phase. Here, we show you how you can quickly change your database to [Amazon Aurora Serverless](#).

The second iteration is to enable [Amazon EC2 Hibernation](#). In a typical development environment, the EC2 instance is only required during business hours. The system should be shut down when it is not in use to reduce cost. However, starting an SAP Commerce system can take a considerable amount of time. Here, we are going to

show you how to enable EC2 Hibernation to take advantage of these cost reductions, while significantly reducing the start-up time.

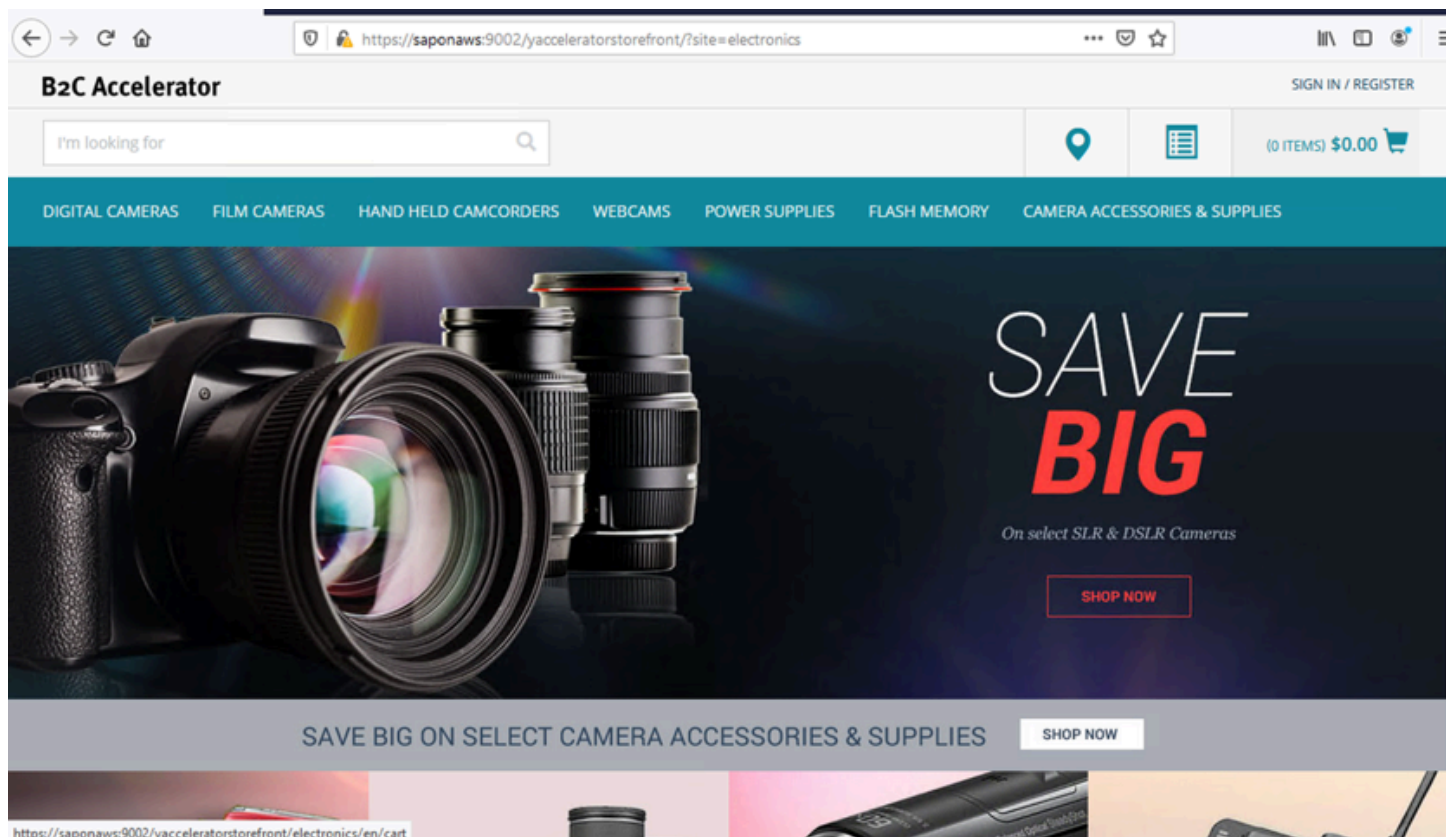
Deploy SAP Commerce

We start by deploying an [AWS CloudFormation](#) template to provision an SAP Commerce on-premises edition running on HSQLDB. As a pre-requisite, you must have an existing VPC and the SAP Commerce software uploaded to an [Amazon S3](#) bucket.

The CloudFormation template we've provided is integrated with the Gradle recipe. These recipes are provided as part of the SAP Commerce software. For more details about the extensions included in each module, refer to the SAP documentation on the Gradle recipe. You can view the YAML template and the detailed deployment guide in our [GitHub repository](#).

One thing to add is that SAP Commerce runs on any operating systems that support the OpenJDK by SAP, called SapMachine. Here, we have selected [Amazon Linux 2](#) as the operating system, simply because Amazon Linux 2 is provided at no additional charge.

Once the CloudFormation template has finished deploying, you can access the site using your web browser. In this case, the site is an electronic store as you can see below. This is part of the B2C accelerator extension in SAP Commerce.



Iteration 1: Change database from HSQLDB to Amazon Aurora Serverless

SAP provides several database options for SAP Commerce. One of the options is to use the SAP HANA database. You can use the AWS [SAP HANA Quick Start](#) to deploy this. Here, we show you how to do this with [MySQL](#)

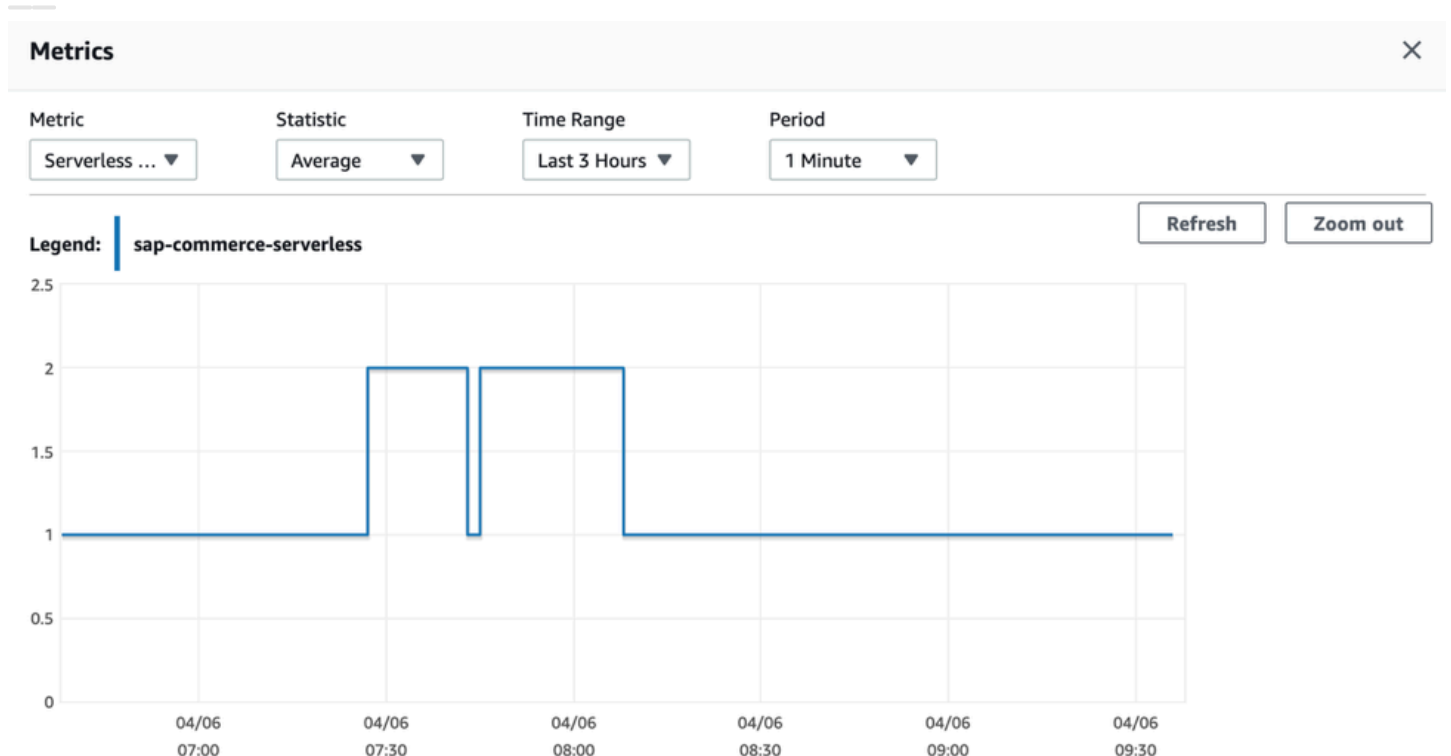
[compatible Amazon Aurora Serverless](#). The beauty of this is that it is cost-effective, fully managed, and the database automatically scales the storage, CPU, and memory up or down based on usage.

You can find the procedure on setting up an Amazon Aurora Serverless database in our AWS documentation. Once you have deployed an Aurora database, there are just two more steps.

1. Download the JDBC driver for MySQL. You can download from [here](#) and then upload to the folder `/usr/sap/hybris/bin/platform/lib/dbdriver/`
2. Update the `local.properties` file in the `/usr/sap/hybris/config` directory.

```
mysql.optional.tabledefs=CHARSET=utf8 COLLATE=utf8_bin
db.driver=com.mysql.cj.jdbc.Driver
db.url=jdbc:mysql://<AmazonAuroraEndpoint>:3306/aurora?verifyServerCertificate=false
db.username=<SAP Commerce user name>
db.password=<SAP Commerce password>
mysql.tabletype=InnoDB
mysql.allow.fractional.seconds=true
mysql.workaround.index.langpk=true
```

The next step is to run the command “ant initialize”. This clears the entire database and then recreates it from scratch. To show the database autoscaling in action, we have included a screenshot of the capacity change when we ran this command. You can see that the serverless database capacity was set to 1 when the database is idle and it automatically changed to 2 when this command ran for about 35 minutes. It then falls back down to 1 shortly after the command has been completed.

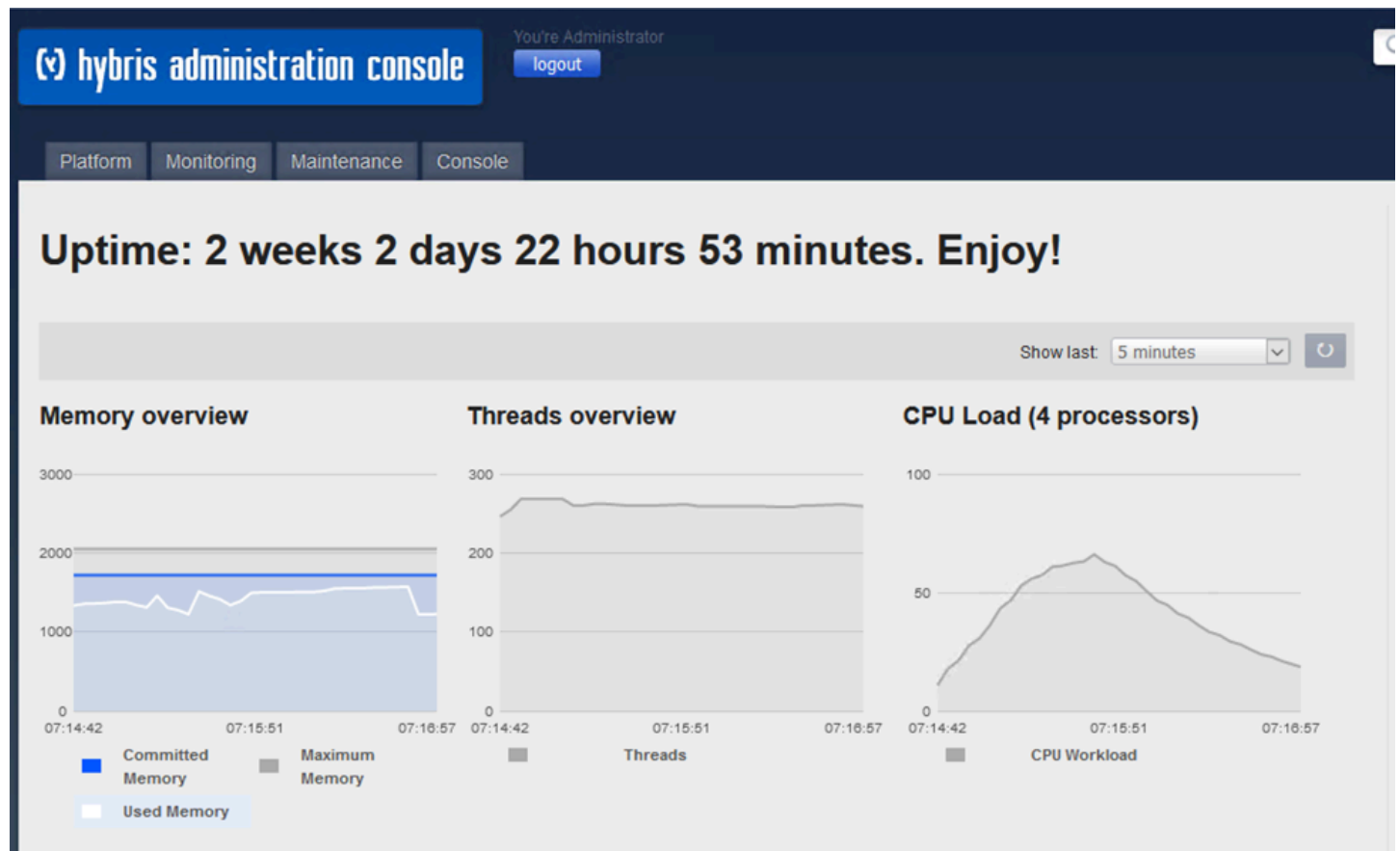


Iteration 2: Enable EC2 Hibernation for faster start uptime

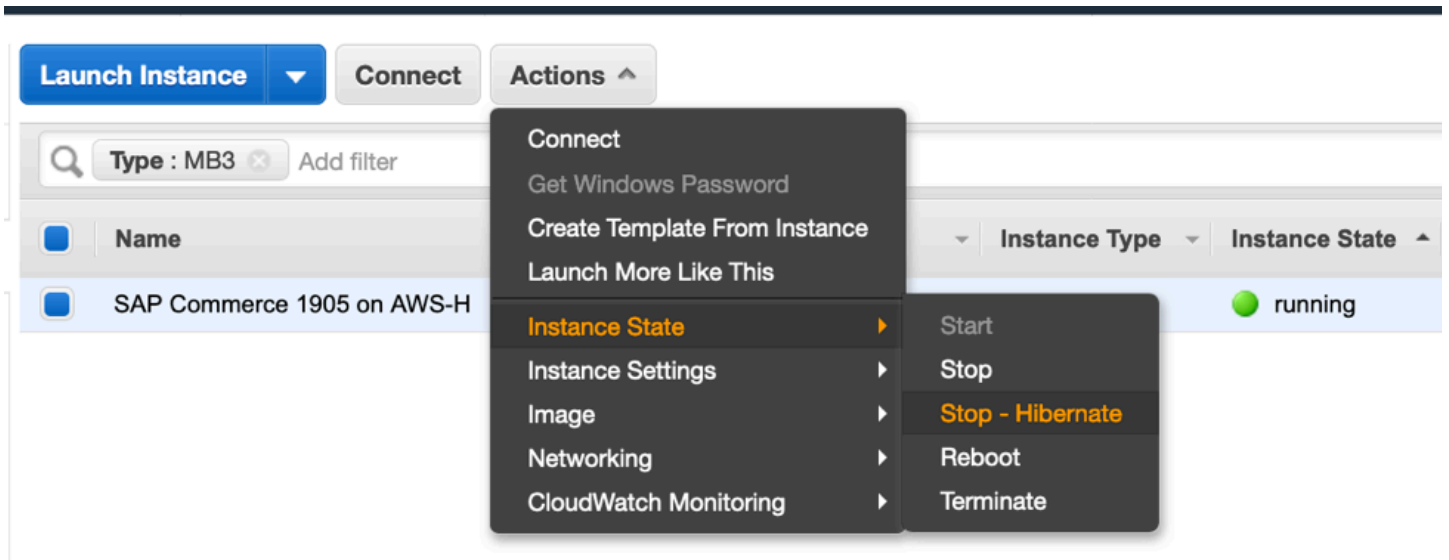
Shutting down your EC2 instances when they are not in use can significantly reduce your costs, especially when you are using this as a development environment. However, one common problem that SAP Commerce developers face is the long start-up time.

One option to reduce the start-up time is to enable EC2 Hibernation. You can find the detailed steps in the [AWS documentation to enable EC2 Hibernation](#). Essentially, you can take an [Amazon Machine Image \(AMI\)](#) of your existing system and launch the EC2 instance with hibernation enabled.

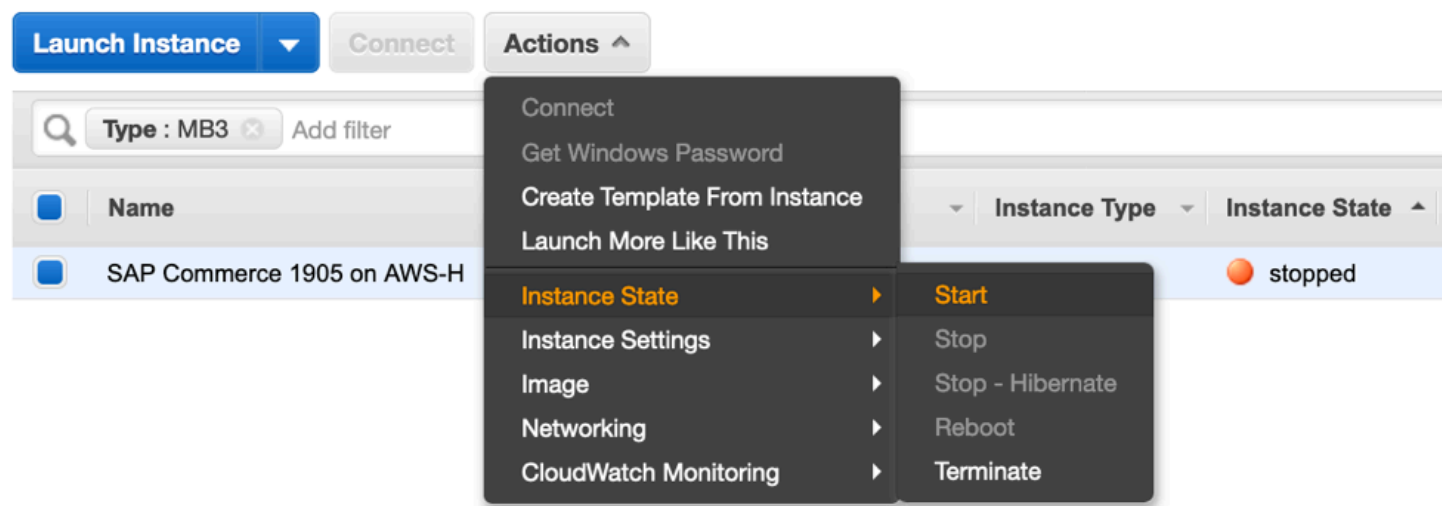
Let's see this in action below. First of all, you can see that the uptime in the Hybris administration console is 2 weeks 2 days 22 hours 53 minutes.



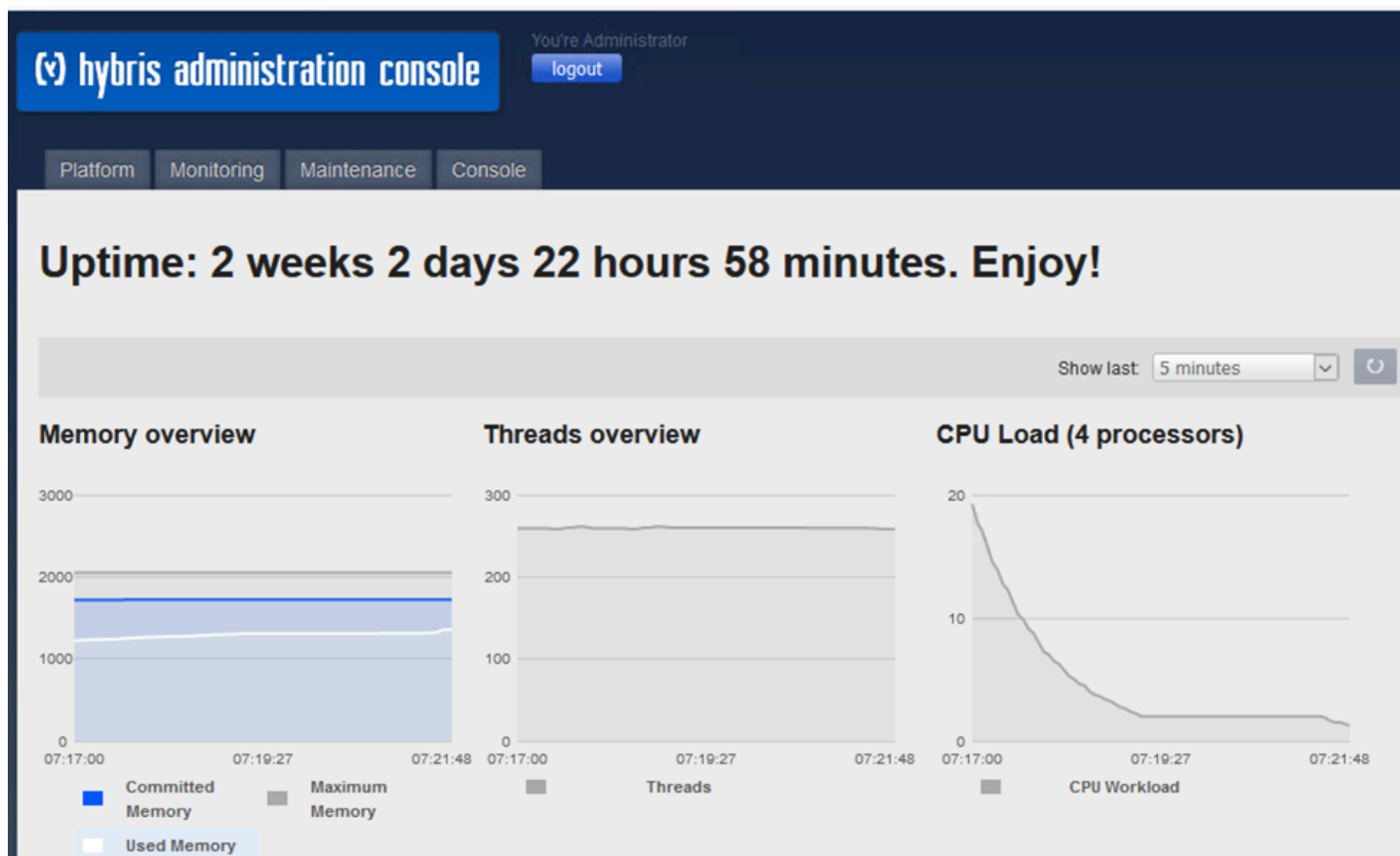
Select the Stop-Hibernate option to reduce cost. This option save the contents from the instance memory (RAM) to the Amazon EBS root volume.



When you start your instance, the RAM contents are reloaded.



You can see that the uptime in the Hybris administration console is 2 weeks 2 days 22 hours 58 minutes. It took less than 3 mins for me to get access to the Hybris administration console after we started the EC2 instance.



That's how easy it is to set all this up! Now you have a development system with minimal effort and investment in infrastructure. Once the development phase is successful, you can again adopt this architecture for your QA and production environment. For example, you can start introducing [Amazon EC2 Auto Scaling](#) or evolve the architecture further to take advantage of [AWS Fargate](#).

Cleanup

Don't forget to delete all the resources created during this walk-through to avoid incurring any more costs:

- CloudFormation Stack
- EC2 Instance
- Amazon S3 bucket
- Amazon Aurora DB instance

Summary

Since we took that first step of building an SAP Commerce system using AWS CloudFormation, we've changed the database to Amazon Aurora Serverless and reduced cost and increased uptime by using Amazon EC2 Hibernation. We've quickly pivoted the architecture with a few clicks of a button. The key message is that it's okay to be less than perfect. AWS gives you the ability to evolve and customize your SAP architecture to your actual needs as they change over time. To me, this is one of the greatest benefits of moving your SAP workload to AWS.

We have used SAP Commerce as an example here, but the concept of an evolutionary architecture can be applied to other SAP products. If you have questions or would like to understand the options for your SAP systems, please contact us at sap-on-aws@amazon.com or visit aws.com/sap to learn more. Start building on AWS today and have fun!

TAGS: [Amazon Aurora](#), [Amazon EC2](#), [Amazon S3](#), [AWS Cloud Formation](#), [SAP Commerce](#), [SAP on AWS](#)